

Final Project

Stevens Institute of Technology

Mauricio Castro

AAI628

Fall 2025

Title:

ECG anomaly detection with CNN autoencoders

Project objective:

The goal of this project is to build an ECG anomaly detection model that is clinically useful, meaning it should prioritize high recall (sensitivity) to minimize false negatives (missed anomalies). The approach used was reconstruction-based anomaly detection with CNN autoencoders trained to reconstruct normal ECG beats; anomalies are identified by higher reconstruction error.

Dataset and evaluation:

Dataset: <http://storage.googleapis.com/download.tensorflow.org/data/ecg.csv> (heartbeat segments of length 140, labeled normal/anomaly).

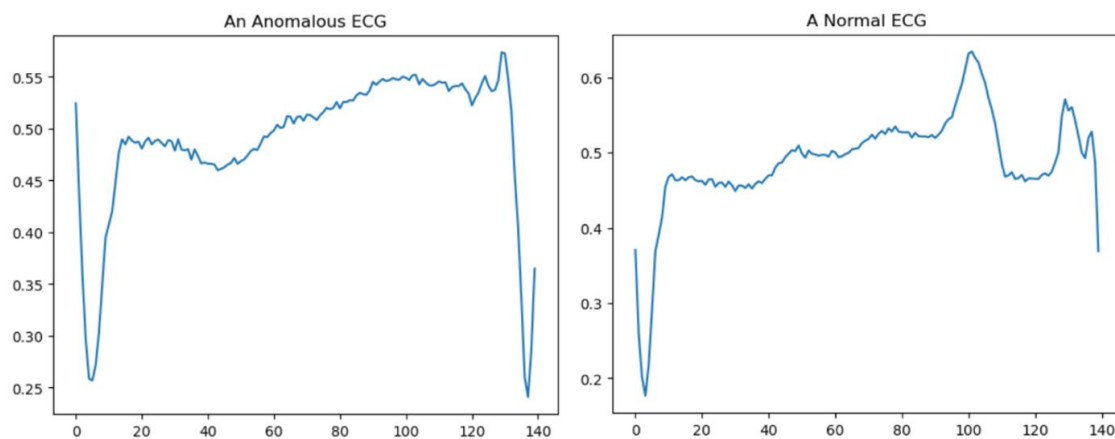


Figure 1a.

Figure 1a. Example normal and anomalous ECG signals from the dataset. Visual differences between classes are often subtle, highlighting the need for automated anomaly detection and expert clinical review.

Evaluation focus: Recall (sensitivity) and False Negatives (FN).

Metrics reported: Accuracy, Precision, Recall, Confusion Matrix [TN FP; FN TP]

Threshold Method Applied Across All Models

For each model, anomaly classification was performed using a model-specific reconstruction error threshold derived from normal training data. After training each autoencoder, reconstruction errors were computed for all normal training samples. The threshold was then defined as:

$$\text{Threshold} = \mu_{\text{train}} + \sigma_{\text{train}}$$

where μ_{train} and σ_{train} represent the mean and standard deviation of reconstruction errors on normal training data.

Each model used a reconstruction error threshold defined as the mean plus one standard deviation of normal training errors, ensuring consistent and fair evaluation across all experiments.

Software Used to Run Models

All models were implemented in Python using TensorFlow/Keras, with NumPy for numerical computation, Scikit-learn for evaluation metrics, and Matplotlib for visualization. Experiments were conducted in a Jupyter Notebook environment.

Hardware

All experiments were conducted on a CPU-based system with the following specifications:

- **Operating System:** Windows 11 Home
- **Processor:** AMD Ryzen 7 7800X3D (8-Core, x64-based processor)
- **Memory:** 64.0 GB RAM
- **Compute Mode:** CPU-only execution

Step-by-step improvement summary table

Model	Main strategy change	Recall	FN	Accuracy	Outcome for clinical sensitivity
1	Baseline CNN AE	0.8822	49	0.901	Baseline
2	+ BatchNorm	0.9712	12	0.952	Major improvement
3	+ Dropout	0.9639	15	0.951	Slight sensitivity drops
4	LeakyReLU + Nadam	0.2909	295	0.665	Failed
5	+ EarlyStopping	0.5529	114	0.774	Still failed sensitivity collapsed
6	Strided CNN + LayerNorm + PReLU + EarlyStopping	0.9832	7	0.945	Best

Final selected model and justification

Selected model: Model 6

It achieved the best clinical objective, highest recall (98.3%) and lowest false negatives (FN=7), meaning it misses the fewest anomalies.

Overall

- **Recall is the key metric** for clinical anomaly detection; accuracy alone is insufficient.
- **Batch Normalization** provided the largest early improvement in sensitivity.
- **Dropout** improves robustness but must be tuned to avoid reducing sensitivity.
- **Optimizer/activation changes** can drastically alter anomaly separability (Model 4).
- **Early stopping** can prevent training from drifting into a low-recall regime (Model 5).
- Architectural choices that preserve temporal structure (Model 6) can outperform pooling-based designs.

Experiment log

Strategies tried, results, lessons learned

Model 1 — Baseline CNN autoencoder

Strategy tried

- Basic CNN autoencoder using Conv1D + MaxPooling in encoder and UpSampling in decoder.
- Output activation: **sigmoid**
- Training: **Adam**, **MAE**, epochs=20, batch=512

Result

- Accuracy = 0.901
- Precision = 0.8801
- Recall = 0.8822
- Confusion Matrix:

$$\begin{bmatrix} 534 & 50 \\ 49 & 367 \end{bmatrix}$$

- FN = 49

Lesson learned

- The baseline model performs reasonably well, but **missed anomalies (FN=49)** are too high for a clinically sensitive system.

Visualization

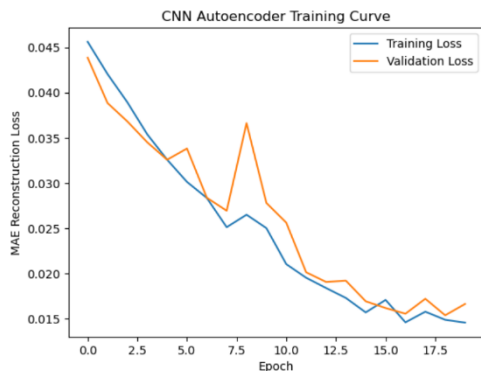


Figure 1. Training Curve Model 1

Figure1. The CNN autoencoder shows stable convergence, with training loss decreasing to 0.0146. Validation loss plateaus around 0.0166, indicating limited generalization capacity and motivating architectural enhancements in subsequent models.

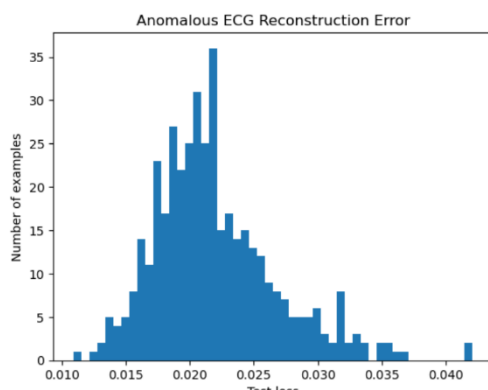


Figure 2.Reconstruction Error Model 1

Figure 2. Most anomalies have low reconstruction errors (~ 0.020), indicating that the baseline autoencoder often reconstructs anomalies too well. This limits anomaly separability and contributes to a high false negative rate.

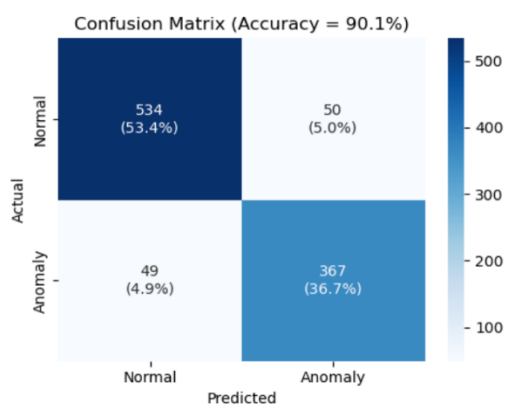


Figure 3.Confusion Matrix Model

Figure 3. The training curve shows stable convergence with early plateau. The reconstruction error histogram reveals low anomaly contrast, and the confusion matrix confirms a high false negative rate (FN = 49), motivating architectural improvements in subsequent models.

Model 2 — Add Batch Normalization

Strategy tried

- Added **BatchNormalization** after each Conv1D layer in encoder and decoder.
- Kept output activation sigmoid.
- Training: **Adam**, **MAE**, epochs=50, batch=128

Result

- Accuracy = 0.952
- Precision = 0.9182
- Recall = 0.9712
- Confusion Matrix:

$$\begin{bmatrix} 548 & 36 \\ 12 & 404 \end{bmatrix}$$

- FN = 12

Lesson learned

- Batch Normalization produced a **major improvement in sensitivity**, reducing missed anomalies from **49 → 12**.

Visualization

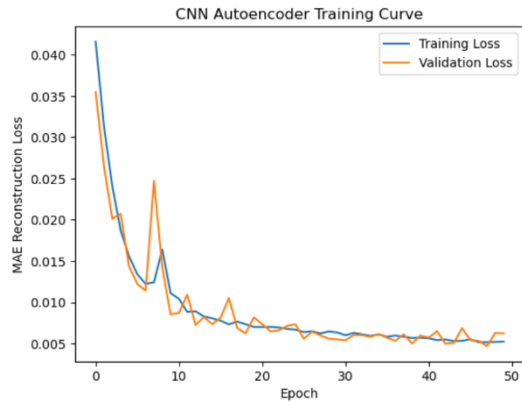


Figure 4. Training Curve Model 2

Figure 4. Batch Normalization improved convergence and generalization, with validation loss stabilizing at 0.0062. This supports the model's improved sensitivity to anomalies.

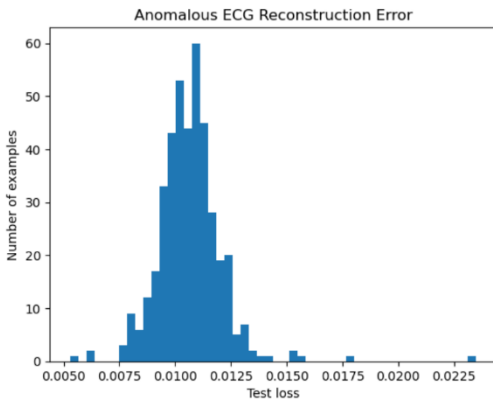


Figure 5. Reconstruction Error Model 2

Figure 5. Compared to Model 1, errors are higher and more spread out, improving anomaly separability and reducing false negatives.

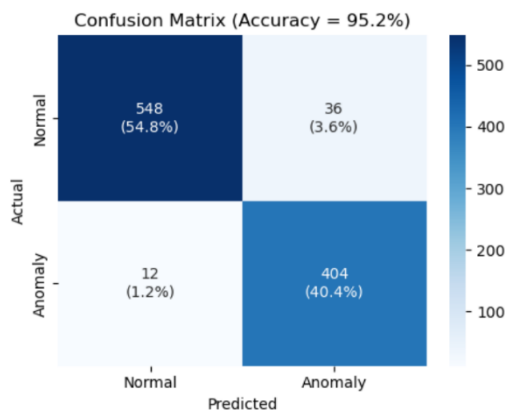


Figure 6. Confusion Matrix Model 2

Figure 6. False negatives dropped from 49 to 12, and recall improved to 97.1%, demonstrating the clinical value of Batch Normalization in anomaly detection.

Model 3 — Add Dropout regularization

Strategy tried

- Added **Dropout** throughout encoder/decoder (encoder 0.2; decoder 0.5 → 0.1).
- BatchNorm + ReLU activations maintained.
- Training: **Adam**, **MAE**, epochs=50, batch=128

Result

- Accuracy = 0.951
- Precision = 0.9218
- Recall = 0.9639
- Confusion Matrix:

$$\begin{bmatrix} 550 & 34 \\ 15 & 401 \end{bmatrix}$$

- FN = 15

Lesson learned

- Dropout improved robustness, but **slightly reduced recall** compared to Model 2 (FN increased **12 → 15**). Too much regularization can reduce sensitivity.
- Model 3 demonstrates that Dropout improves robustness but introduces a small sensitivity trade-off, slightly increasing false negatives compared to Model 2.

Visualization

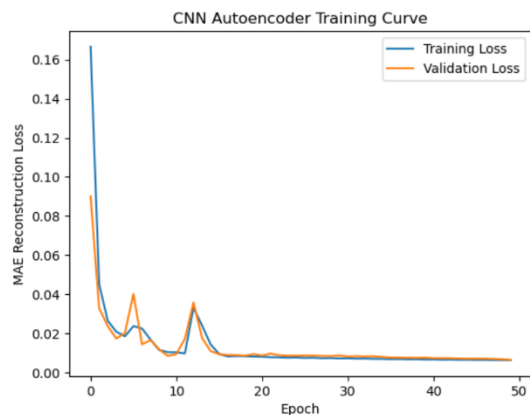


Figure 7.Training Curve Model 3

Figure 7. Dropout introduces mild variability in validation loss while maintaining stable convergence.

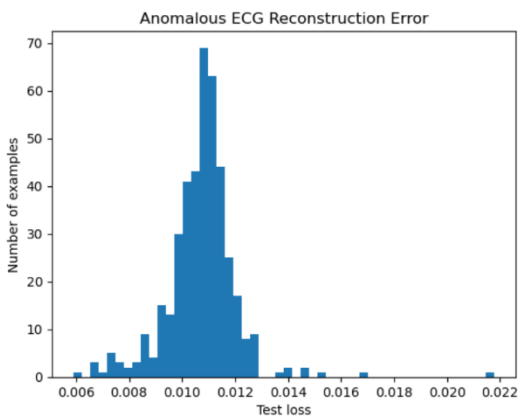


Figure 8. Reconstruction Error Model 3

Figure 8. Dropout improves robustness but slightly reduces anomaly separation, contributing to a small increase in false negatives.

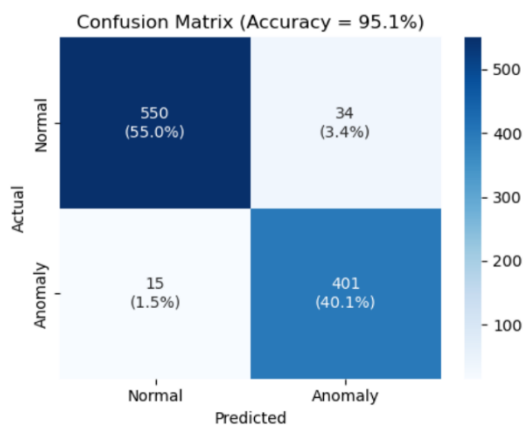


Figure 9.Confusion Matrix Model 3

Figure 9. The model maintains high recall (96.4%) with a small increase in false negatives compared to Model 2, indicating a trade-off introduced by Dropout regularization.

Model 4 — LeakyReLU + Nadam (performance collapse)

Strategy tried

- Replaced ReLU with **LeakyReLU ($\alpha=0.1$)** everywhere.
- Changed optimizer to **Nadam ($lr=1e-3$)**.
- Linear output for reconstruction.
- Training: epochs=50, batch=128

Result

- Accuracy = 0.665
- Precision = 0.7516
- Recall = 0.2909
- Confusion Matrix:

$$\begin{bmatrix} 544 & 40 \\ 295 & 121 \end{bmatrix}$$

- FN = 295

Lesson learned

- This configuration was **clinically unacceptable**. The model likely became too flexible/generalized and failed to separate anomalies via reconstruction error, causing **recall collapse**.
- Model 4 demonstrates that increased model flexibility without proper training control can lead to over-generalization, severely degrading anomaly sensitivity despite low reconstruction loss.

Visualization

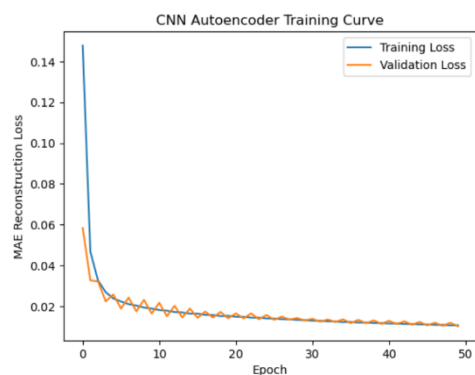


Figure 10. Training Curve Model 4

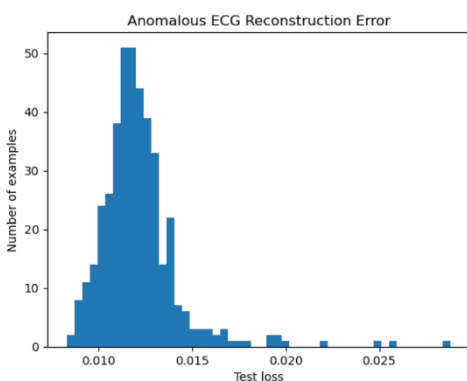


Figure 11. Reconstruction Error Model 4

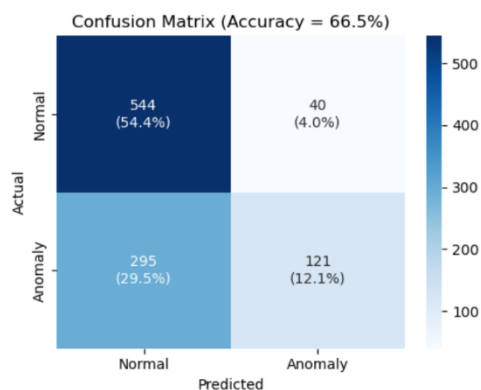


Figure 12. Confusion Matrix Model 4

Figure 10. Despite smooth convergence and low reconstruction loss, the model failed to separate anomalies, demonstrating that low loss alone does not guarantee clinical sensitivity.

Figure 11. Most anomalies have low reconstruction errors (~ 0.011), indicating that the model reconstructs them too accurately. This over-generalization severely reduces anomaly separability and contributes to the high false negative rate.

Figure 12. Model 4 missed 295 anomalies, reducing recall to around 29.1%. This confirms that architectural changes without training control can severely degrade clinical performance.

Model 5 — Add Early Stopping to control training dynamics

Strategy tried

- Kept LeakyReLU + Nadam + Dropout + linear output.
- Added **EarlyStopping** (monitor val_loss, patience=5, restore_best_weights=True).
- Training: increased to 100 epochs (stopped at 70), batch=128

Result

- Accuracy: 0.774
- Precision: 0.8519
- Recall: 0.5529
- Confusion Matrix:

$$\begin{bmatrix} 534 & 50 \\ 114 & 302 \end{bmatrix}$$

- FN = 114

Lesson learned

- Although EarlyStopping reduced excessive training duration, it did not fully prevent over-generalization under heavy regularization.
- The model continued to reconstruct anomalous ECGs with low error, resulting in insufficient anomaly separability and a high false negative rate. This testing demonstrates that training control alone is insufficient when combined with aggressive dropout and highly flexible activations.

Visualization

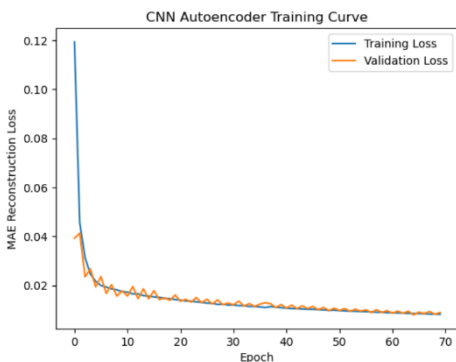


Figure 13. Training Curve Model 5

Figure 13. EarlyStopping halted training at epoch 62. Despite smooth convergence, validation loss plateaued, suggesting limited benefit from continued training.

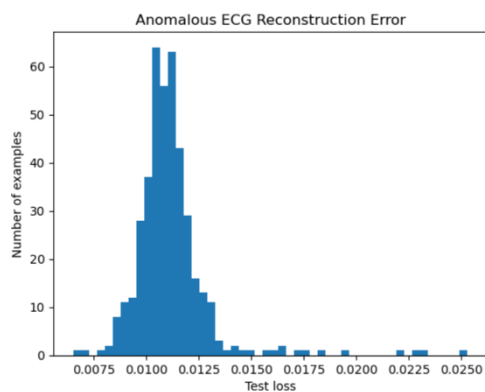


Figure 14. Reconstruction Error Model 5

Figure 14. Most anomalies exhibit low reconstruction error (~ 0.010 – 0.0125), indicating continued over-generalization and poor anomaly separability

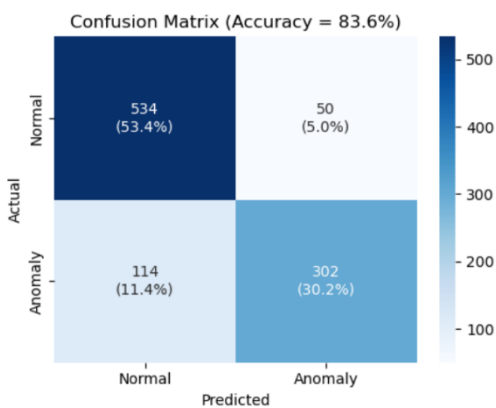


Figure 15. Confusion Matrix Model

Figure 15. The model missed 114 anomalies, resulting in a recall of 55.3%. This confirms that EarlyStopping alone was insufficient to restore clinical sensitivity.

Model 6 — Strided convolutions + transposed convolutions + LayerNorm + PReLU

Strategy tried

- Replaced MaxPooling/UpSampling with:
 - Encoder: **Conv1D with stride=2**
 - Decoder: **Conv1DTranspose with stride=2**
- Used **LayerNormalization** instead of BatchNorm.
- Used **PReLU** activation (learnable).
- Linear output.
- Optimizer: **Adam, MAE**, same as model 4
- Training: decreased to 40 from 100 epochs (stopped at 18), batch = 512
- **EarlyStopping** (monitor val_loss, patience=3, restore_best_weights=True).

Result

- Accuracy = 0.945
- Precision = 0.8950
- Recall = 0.9832
- Confusion Matrix:

$$\begin{bmatrix} 536 & 48 \\ 7 & 409 \end{bmatrix}$$

- FN = 7

Lesson learned

- This model achieved the highest recall and lowest false negatives. Strided CNN encoding/decoding plus LayerNorm and PReLU improved anomaly sensitivity while keeping performance stable.
- Model 6 achieved the highest clinical sensitivity by combining architectural refinement and training control, reducing false negatives to 7 and maximizing anomaly separability.

Visualization

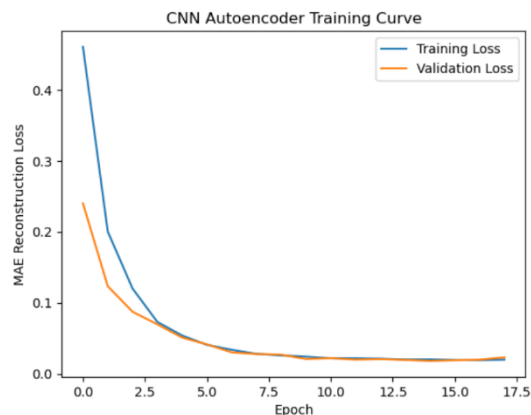


Figure 16. Training Curve Model 6

Figure 16. The model converged smoothly with stable validation loss, indicating effective training dynamics and generalization.

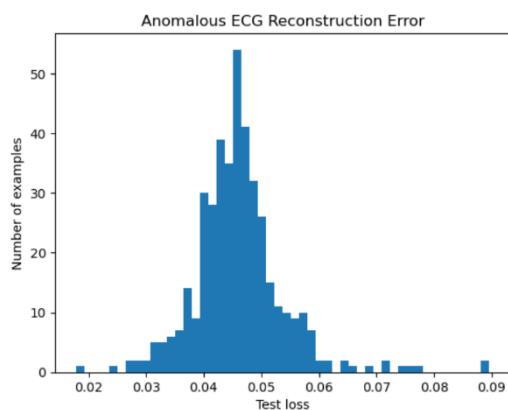


Figure 17. Reconstruction Error Model 6

Figure 17. Most anomalies exhibit high reconstruction error (~ 0.045), confirming strong anomaly contrast and separability.

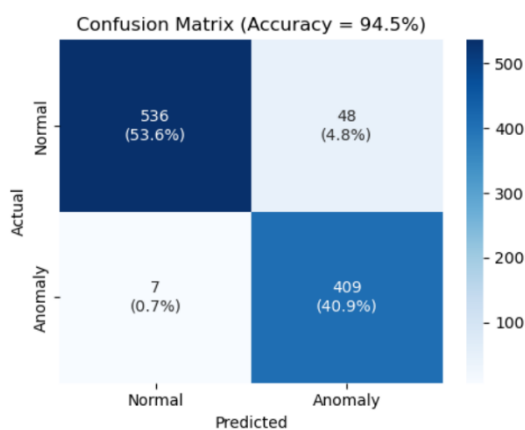


Figure 18. Confusion Matrix Model 6

Figure 18. False negatives dropped to 7, and recall reached 98.3%, confirming the effectiveness of Layer Normalization and strided convolution in restoring anomaly sensitivity.

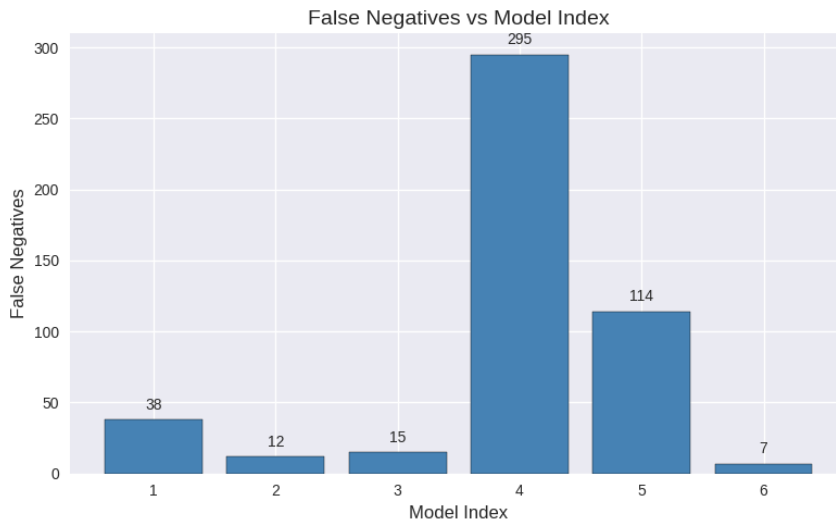


Figure 19. False negatives across Models 1–6. The chart highlights the progression of anomaly sensitivity, with Model 6 achieving the lowest false negative count (7) and Model 4 demonstrating the impact of over-generalization.

Future improvements for Model 6

- Evaluate Model 6 on a different ECG dataset to assess generalization performance and robustness across varying signal characteristics and acquisition conditions.
- Leverage GPU acceleration to significantly reduce training time, enabling larger batch sizes, longer training schedules, and more extensive hyperparameter exploration.
- Explore more advanced architectures, such as Variational Autoencoders (VAEs), attention-based autoencoders, or hybrid CNN–Transformer models, to capture complex temporal dependencies and improve anomaly separability.
- Extend evaluation to real-time or streaming scenarios, assessing feasibility for continuous monitoring applications with human-in-the-loop supervision.

Summary

Across six CNN autoencoder models, this study shows that anomaly detection performance depends on more than minimizing reconstruction loss. Increasing model capacity and flexibility can recover learning, but also introduces the risk of over-generalization, where anomalous ECGs are reconstructed too accurately and false negatives increase. Models 4 and 5 demonstrated that low loss and stable training curves do not guarantee clinical sensitivity.

The final model achieved the best running by addressing the root cause through architectural improvement, specifically combining Layer Normalization with strided convolution. This approach preserved anomaly-relevant features, stabilized training, and produced the lowest false negative rate. However, regardless of performance, even the most accurate models require human supervision for final decision-making due to the sensitivity and clinical risk associated with missed anomalies. These models are best placed as decision-support tools, augmenting expert judgment rather than replacing it.